

Phase 9 Implementation Plan — PLN, Special Expenses, Budget Pace, Monthly History, Encryption at Rest

For agentic workers: REQUIRED SUB-SKILL: Use `superpowers:subagent-driven-development` (recommended) or `superpowers:executing-plans` to implement this plan task-by-task. Steps use checkbox (`- [ ]`) syntax for tracking.

Goal: Ship the five approved Phase 9 features of the Trim budgeting app: PLN currency, opt-in special expenses, a budget-pace number, a monthly spending history, and at-rest encryption of users' financial data.

Architecture: All data flows client -> Express API -> Supabase (client never talks to Supabase for data). Every money computation is JS-side in Express, which is what makes Task 5's encrypt-on-write/decrypt-after-fetch design safe. Spec: `docs/superpowers/specs/2026-07-17-pln-privacy-history-pace-special-design.md` — read it first; decisions there are locked.

Tech Stack: React 18 + Vite + TanStack Query v5 + Tailwind (client); Express 4 ESM + Zod + supabase-js (server); Supabase Postgres; `node:crypto` + `node:test` (Task 5 only, built into Node 20 — no new dependencies).

Global Constraints

- **One task per session** (project rule). Each task below = one build session = one of BUILD_PLAN.md Phase 9's tasks 9.1–9.5.
- **Definition of done, every task:** `npm run build` passes in `client/`; server starts cleanly; feature clicked-to in the running UI (use the `launch.json` dev servers, never Bash); FEATURES.md + BUILD_PLAN.md updated (SECURITY.md too in Task 5); committed; state explicitly that the branch still needs merging to main.
- **Tone rules:** never red for user behaviour (rose-400 soft warning max); friendly copy; 3-tap logging path untouched.
- **Currency:** always read from `/api/me` preferences; never hardcode. No FX.
- **Migrations:** run against Supabase project (ref in memory: `project_supabase_operations.md`) via MCP `apply_migration` or give Alex exact dashboard SQL steps. Files also land in `server/migrations/` for fresh-project rebuilds.
- **Numbers 010–013 belong to Phase 9.** Phase 8 (unbuilt bank-sync spec) referenced 010/011 — its DDL becomes 014+ (note added to BUILD_PLAN.md in Task 1).
- **server/scripts/devMock.js** must mirror every API-shape change (new fields/params) so `npm run dev:mock` stays a faithful UI dev harness.
- Server validates amounts as positive, finite, $\leq 1,000,000,000$ — unchanged by encryption (validation happens at the API boundary, pre-encryption).

Task 1: PLN currency (BUILD_PLAN 9.1)

Files: - Create: `server/migrations/010_pln_currency.sql` - Modify: `server/routes/me.js:9`, `server/lib/parser.js:9,26,43–46`, `client/src/lib/format.js:1–6`, `client/src/pages/Settings.js:26`, BUILD_PLAN.md (Phase 8 note + 9.1 checkbox), FEATURES.md (Settings + Money model lines)

Interfaces: - Produces: 'PLN' as a valid currency_code enum value, preferences.currency value, and formatMoney(n, 'PLN') rendering zł via pl-PL.

□ **Step 1: Write the migration**

```
-- server/migrations/010_pln_currency.sql
-- Phase 9.1: Polish złoty. ADD VALUE must not share a transaction with
-- statements that USE the value – keep this file to this single statement.
alter type public.currency_code add value if not exists 'PLN';
```

- **Step 2: Apply it to Supabase** — MCP apply_migration (name pln_currency), or give Alex the SQL to paste into the dashboard SQL editor. Verify: select unnest(enum_range(null::public.currency_code)); returns 5 rows including PLN.

□ **Step 3: Server accepts PLN** — server/routes/me.js:9:

```
currency: z.enum(['GBP', 'USD', 'AUD', 'VND', 'PLN']).optional(),
```

- **Step 4: Parser knows PLN** — server/lib/parser.js: line 9 same enum addition as Step 3; line 26 becomes "currency": "GBP" | "USD" | "AUD" | "VND" | "PLN"; in the cue rules (~line 43) add:

– "zł", "złoty", "złotych", "pln" -> PLN ("50 zł" -> 5000 – grosz minor units, ×100 like pence)

and update the fallback line to Any currency outside GBP/USD/AUD/VND/PLN, or no cue at all -> use the user's default.

- **Step 5: Client formats PLN** — client/src/lib/format.js add PLN: 'pl-PL', to CURRENCY_LOCALE; client/src/pages/Settings.jsx add { code: 'PLN', label: 'PLN · Polish Złoty' }, to CURRENCIES (match existing label style, line 22-25).

- **Step 6: Verify** — cd client && npm run build (passes); start dev servers (launch.json / preview tool); in Settings pick "PLN · Polish Złoty"; Dashboard hero renders zł amounts (pl-PL puts zł after the number, comma decimals). Type-it-instead check (needs real server + ANTHROPIC_API_KEY): "spent 50 zł on lunch" parses to 50.00. With mock server, skip the parse check.

- **Step 7: Docs + commit** — BUILD_PLAN.md: tick 9.1 (the Phase 8 migration-renumbering note was already added in the 2026-07-17 planning session). FEATURES.md: Settings currency picker line + Money model gain PLN. Commit: git add -A && git commit -m "9.1: add PLN currency (enum, parser cues, pl-PL formatting)".

Task 2: Opt-in special expenses (BUILD_PLAN 9.2)

Files: - Create: server/migrations/011_special_expenses.sql - Modify (server): routes/me.js, routes/transactions.js, routes/budgets.js, routes/dashboard.js, routes/projections.js, routes/affordability.js, routes/wins.js, routes/analytics.js, lib/askContext.js, scripts/devMock.js - Modify (client): pages/Settings.jsx, components/QuickAddDialog.jsx, pages/Transactions.jsx, pages/Dashboard.jsx - Docs: FEATURES.md, BUILD_PLAN.md

Interfaces: - Produces (server): preferences.specialExpensesEnabled: boolean on GET/PATCH /api/me and GET /api/dashboard; isSpecial: boolean accepted by POST/PATCH /api/transactions and returned on every transaction row (is_special in DB, isSpecial unused — rows return raw is_special, matching existing snake_case row passthrough); month.specialThisMonth: number on /api/dashboard; special: number per series bucket on /api/analytics. - Produces (shared helper): excludeSpecial(txRows, enabled) — see Step 3. - Consumes: nothing from other tasks.

□ **Step 1: Migration**

```
-- server/migrations/011_special_expenses.sql
-- Phase 9.2: opt-in special expenses (outside the monthly budget).
alter table public.transactions
  add column is_special boolean not null default false;
alter table public.user_stats
  add column special_expenses_enabled boolean not null default false;
```

Apply via MCP apply_migration (name special_expenses). Verify: select is_special from public.transactions limit 1; runs.

- **Step 2: Preference plumbing** — server/routes/me.js: add to prefsSchema: specialExpensesEnabled: z.boolean().optional();; in PATCH payload mapping add if (parsed.data.specialExpensesEnabled !== undefined) payload.special_expenses_enabled = parsed.data.specialExpensesEnabled;; add specialExpensesEnabled: stats.special_expenses_enabled, to both GET and PATCH preferences responses (PATCH's .select(...) must add special_expenses_enabled). Also add the same field to the preferences block of routes/dashboard.js (line ~143).

- **Step 3: Shared exclusion helper** — the “dormant rule” in one place. Create in server/lib/gamification.js? No — money math, not gamification. Add to a new tiny module server/lib/special.js:

```
// Phase 9.2: special expenses are excluded from budget math ONLY while the
// user's opt-in preference is on. Off = flags dormant, everything counts.
export function excludeSpecial(rows, specialEnabled) {
  return specialEnabled ? rows.filter((t) => !t.is_special) : rows;
}

export function sumSpecial(rows, specialEnabled) {
  if (!specialEnabled) return 0;
  return rows.reduce(
    (sum, t) => (t.is_special && t.type !== 'income' ? sum + Number(t.amount) : sum),
    0,
  );
}
```

- **Step 4: Transactions route** — server/routes/transactions.js:
- Both Zod schemas gain isSpecial: z.boolean().optional(),.
 - GET select gains is_special (line 40).
 - POST: after the category-type check, guard if (parsed.data.isSpecial && type

=== 'income') return res.status(400).json({ error: 'Only expenses can be special' }); and add is_special: parsed.data.isSpecial ?? false, to the insert object.

- PATCH: map isSpecial -> is_special in the update payload with the same income guard (fetch the row's type first — the PATCH handler already loads the transaction to authorise it; if it doesn't, add .select('type') to the ownership check).
- Also add ?month=YYYY-MM support **now** (Task 4 consumes it): `const month = /^\\d{4}-\\d{2}$/.test(req.query.month ?? '') ? req.query.month : null;` and when set, `.gte('date', ${month}-01).lt('date', nextMonth-FirstISO(month))` with

```
function nextMonthFirstISO(ym) {
  const [y, m] = ym.split('-').map(Number);
  return new Date(Date.UTC(y, m, 1)).toISOString().slice(0, 10);
}
```

- **Step 5: Budget-math exclusions (server)** — each route below needs `special_expenses_enabled`; where `user_stats` isn't already fetched, add it to the existing `Promise.all(supabase.from('user_stats').select(...))` in these routes.

- `budgets.js` GET (~line 51): `const countable = excludeSpecial(txRes.data, specialEnabled);` and build `spendByCat` from `countable`.
- `dashboard.js`: keep hero totals from ALL rows (income/expenses loop unchanged); build `categoryTotals` (donut, top-5, alerts) from `excludeSpecial(txs, specialEnabled)`; add to the month response object: `specialThisMonth: Number(sumSpecial(txs, specialEnabled).toFixed(2)),`
- `projections.js`: it fetches only amount — `select amount, is_special;` wrap both month arrays with `excludeSpecial(..., specialEnabled)` before summing (needs the new `user_stats` fetch — reuse it in Task 3).
- `affordability.js`: `select amount, category_id, is_special;` `spendByCat/totalSpent` built from `excludeSpecial(txRes.data, specialEnabled)`.
- `wins.js`: `user_stats` `select gains special_expenses_enabled;` `expensesByCategory` loop skips `tx.is_special` rows when enabled (`tx` `select gains is_special`).
- `analytics.js`: `tx` `select gains is_special;` series buckets `gain special: 0;` in the accumulation loop, expenses stay cash-flow (include everything) and additionally if `(t.is_special && specialEnabled)` `bucket.special += amount;`; round special in the final pass.
- `lib/askContext.js`: `loadAskContext tx` `select gains is_special;` in `buildAskContext`, recent transactions map gains if `(t.is_special)` `out.special = true;`; the returned object gains `specialExpensesEnabled: !!stats?.special_expenses_enabled,` and, when enabled, a `specialThisMonthTotal` computed like `sumSpecial`.

- **Step 6: devMock mirrors** — `server/scripts/devMock.js`: seed a couple of `is_special: true` expenses, accept `isSpecial` on create/patch, add `specialExpensesEnabled` to its `/api/me` + dashboard preferences, `specialThisMonth` to dashboard month, `special` to analytics buckets, `?month=` filter on transactions. Keep shapes identical to real routes.

- **Step 7: Settings toggle (client)** — `pages/Settings.jsx`: state `const [specialEn-`

abled, setSpecialEnabled] = useState(false);, seed from data.preferences.specialExpense saveSpecialEnabled(next) mutating { specialExpensesEnabled: next }. Render a toggle row below Simple mode, copying the exact simple-mode switch markup (lines ~160-176) with label **“Special expenses”** and helper text *“Track gifts, trips and one-offs outside your monthly budget. Off by default — flip it on and a star appears in Quick-Add.”*

- **Step 8: Quick-Add toggle (client)** — components/QuickAddDialog.jsx: needs specialExpensesEnabled — the dialog already queries nothing itself for prefs? It receives simpleMode from its parent; pass specialEnabled the same way from App.jsx/Dashboard.jsx (wherever simpleMode comes from — follow that exact prop path). Add state const [isSpecial, setIsSpecial] = useState(false); (reset alongside other fields on close). Inside the showMore advanced block (line ~431), after the Note input, render only when specialEnabled && type === 'expense':

```
<label className="flex items-center justify-between rounded-lg border border-border/60"
  <span className="flex items-center gap-2 text-sm">
    <Star className="h-4 w-4 text-amber-400" aria-hidden />
    Special expense
    <span className="text-xs text-muted-foreground">kept out of your monthly budget</span>
  </span>
  <input
    type="checkbox"
    className="h-4 w-4 accent-primary"
    checked={isSpecial}
    onChange={(e) => setIsSpecial(e.target.checked)}
  />
</label>
```

(Star from lucide-react.) Include isSpecial in the mutation payload only when type === 'expense'. Success toast when special: toast.success('Logged as special (star)', { description: 'Outside your monthly budget' }) in place of the plain “Logged” branch (keep XP/level branches above it untouched).

- **Step 9: Transactions page (client)** — pages/Transactions.jsx:
 - Read specialExpensesEnabled from the existing me query (line 161).
 - Row star marker: next to the amount, {me?.preferences?.specialExpensesEnabled && tx.is_special ? <Star className="h-3.5 w-3.5 text-amber-400" aria-label="Special expense" /> : null}.
 - One-tap star/unstar (the retroactive exclude button): a ghost icon button beside the Pencil (line ~401), expenses only:

```
{me?.preferences?.specialExpensesEnabled && tx.type === 'expense' ? (
  <Button
    variant="ghost"
    size="icon"
    aria-label={tx.is_special ? 'Unmark special' : 'Mark as special'}
    onClick={() => updateMutation.mutate({ id: tx.id, payload: { isSpecial: !tx.is_spec
  >
    <Star className={cn('h-3.5 w-3.5', tx.is_special ? 'fill-amber-400 text-amber-400'
  </Button>
```

) : `null`}

- `AlertDialog` (line 49): add the same checkbox row as Step 8 (pass `specialEnabled` down), seeding from `tx.is_special`, sending `isSpecial` in `onSave`'s payload for expenses.
 - Filter chip: alongside the existing type filter, when the pref is on add a “(star) Special” toggle chip filtering `t.is_special` client-side.
- **Step 10: Dashboard chip (client)** — `pages/Dashboard.jsx`: pass `specialThisMonth={month.specialThisMonth ?? 0}` and the pref into `HeroBalance` (line 84); inside, beside the In/Out chips (lines ~120-141), when `specialThisMonth > 0` render a third chip in the same markup style: label `(star) Special`, value `formatMoney(specialThisMonth, currency)`, amber accent (`text-amber-400`).
- **Step 11: Verify** — client build passes; dev server click-through: enable the toggle in Settings -> Quick-Add advanced area shows the star toggle -> log a special expense -> Dashboard: hero Out includes it, by-category card and budget bars do NOT, (star) chip shows the amount -> Transactions: row shows star, one-tap unstar returns it to the budget (watch a budget bar move) -> disable the pref in Settings: all special UI disappears and the expense counts normally again. Also confirm income rows never show a star action.
- **Step 12: Docs + commit** — `FEATURES.md`: new “Special expenses (opt-in)” subsection under `Pages/Dashboard + Settings` line + Money model note; `BUILD_PLAN.md` tick 9.2. Commit 9.2: opt-in special expenses – flag, exclusions, star UI.
-

Task 3: Budget pace (BUILD_PLAN 9.3)

Files: - Modify: `server/routes/projections.js`, `server/scripts/devMock.js`, `client/src/component client/src/components/SimpleMonthCard.jsx`, `client/src/pages/Dashboard.jsx` (only if prop threading is needed), `FEATURES.md`, `BUILD_PLAN.md`

Interfaces: - Consumes: Task 2's `excludeSpecial + special_expenses_enabled` fetch in `projections.js`. - Produces: GET `/api/projections/month` gains top-level pace: { target: number, spent: number, delta: number } | null, present on **both** ready: true and ready: false responses.

- **Step 1: Server** — `server/routes/projections.js`: add `user_stats(select('simple_mode, monthly_limit, special_expenses_enabled'))` to the `Promise.all` (Task 2 already needs it — reuse). After computing `spendSoFar` (post-exclusion) and `monthlyBudget`, define:

```
const stats = statsRes.data;
const budgetSource = stats?.simple_mode && stats?.monthly_limit !== null
  ? Number(stats.monthly_limit)
  : monthlyBudget; // sum of monthly budgets, or null
const pace = budgetSource === null || budgetSource <= 0
  ? null
  : {
    target: Number(((budgetSource * daysElapsed) / daysInMonth).toFixed(2)),
    spent: Number(spendSoFar.toFixed(2)),
```

```

    delta: Number(((budgetSource * daysElapsed) / daysInMonth - spendSoFar).toFixed(2));
  };

```

Add pace to BOTH res.json branches — the cold-start early return (line 92-98) must compute spendSoFar first, so hoist the spendSoFar reduction above the guard (it already is — line 83). Mirror pace in devMock.js's projections payload.

- **Step 2: Verify server** — `curl -s localhost:3001/api/projections/month -H "Authorization: Bearer <token>"` (or mock server without auth): response contains `pace.target ≈ budget × dayOfMonth ÷ daysInMonth`. With no budgets and `simple_mode` off: `pace: null`.

- **Step 3: Pace line in AffordabilityCheck** — `components/AffordabilityCheck.jsx`: add

```

const { data: proj } = useQuery({
  queryKey: ['projections', 'month'],
  queryFn: () => api.get('/api/projections/month'),
});
const pace = proj?.pace ?? null;

```

(query key matches MonthProjection's — cache is shared, no extra request). Render between the header row and the amount input, only when `pace !== null`:

```

{pace ? (
  <p className="mt-2 text-xs text-muted-foreground">
    {pace.delta >= 0 ? (
      <span className="text-primary">OK</span>
    ) : (
      <span className="text-amber-400">!  

    )}{' '}
    By day {proj.daysElapsed}, about {formatMoney(pace.target, currency)} of your budget  

    would typically be used — you're at{' '}
    <span className={cn('nums font-medium', pace.delta >= 0 ? 'text-primary' : 'text-amber-400')}>
      {formatMoney(pace.spent, currency)}
    </span>
    {pace.delta < 0 ? ' — a touch ahead of pace, plenty of month left.' : '.'}
  </p>
) : null}

```

Amber, never red/rose (tone rule).

- **Step 4: Simple mode** — `components/SimpleMonthCard.jsx`: same `useQuery` + the same paragraph (identical JSX, placed under the progress bar, line ~107). Simple-mode users get pace against `monthly_limit` because the server already chose the budget source.
- **Step 5: Verify UI** — dev server: normal mode Dashboard shows the pace line inside “Can I afford this?” with today's day-of-month; flip Settings -> Simple mode: `SimpleMonthCard` shows the same line vs the single limit; delete all budgets (normal mode, no limit): line disappears. Client build passes.
- **Step 6: Docs + commit** — `FEATURES.md` Dashboard section: pace line description; `BUILD_PLAN` tick 9.3. Commit 9.3: budget pace — “what should I have spent by

now".

Task 4: Monthly history (BUILD_PLAN 9.4)

Files: - Create: `client/src/components/MonthlyHistory.jsx` - Modify: `client/src/pages/Analytics.jsx`, `client/src/pages/Transactions.jsx`, `server/scripts/devMock.js` (only if its analytics window < 24 months), `FEATURES.md`, `BUILD_PLAN.md`

Interfaces: - Consumes: `/api/analytics?months=24` (`series[]` {`ym`, `label`, `income`, `expenses`, `net`, `special`} — special from Task 2); `/api/transactions?month=YYYY-MM` (from Task 2 Step 4). - Produces: route `/transactions?month=YYYY-MM` deep-linkable (URL param seeds the month filter).

- **Step 1: Analytics fetches 24 months** — `pages/Analytics.jsx:33–35`: change to `queryKey: ['analytics', 24]`, `api.get('/api/analytics?months=24')`. The 6-month chart keeps its shape with `const chartSeries = (data?.series ?? []).slice(-6)`; — swap the chart's data source to `chartSeries`. (mom and topCategories are unaffected by the wider window.)
- **Step 2: MonthlyHistory component** — create `client/src/components/MonthlyHistory.jsx`:

```
import { Link } from 'react-router-dom';
import { Star } from 'lucide-react';
import { Card, CardContent } from '@components/ui/card';
import { formatMoney } from '@lib/format';
import { cn } from '@lib/utils';

/** Phase 9.4 – one row per month, newest first; rows link to the
 * Transactions page filtered to that month. */
export function MonthlyHistory({ series, currency, showSpecial }) {
  const months = [...series]
    .filter((s, i, arr) => {
      const firstWithData = arr.findIndex((m) => m.income > 0 || m.expenses > 0);
      return firstWithData !== -1 && i >= firstWithData; // trim pre-signup months
    })
    .reverse();
  if (months.length === 0) return null;
  const thisYm = new Date().toISOString().slice(0, 7);

  return (
    <Card className="lift border-border/60 bg-card/70 backdrop-blur">
      <CardContent className="p-5 sm:p-6">
        <h3 className="text-xs font-semibold uppercase tracking-[0.16em] text-muted-foreground">
          Monthly history
        </h3>
        <div className="mt-3 divide-y divide-border/60">
          {months.map((m) => (
            <Link
```



```

    key={m.ym}
    to={` /transactions?month=${m.ym}`}
    className="flex items-center justify-between gap-3 py-2.5 text-sm transit
  >
  <span className="w-24 shrink-0 font-medium">
    {m.label} {m.ym.slice(0, 4)}
    {m.ym === thisYm ? (
      <span className="ml-1 text-[10px] text-muted-foreground">so far</span>
    ) : null}
  </span>
  <span className="nums flex-1 text-right text-muted-foreground">
    -{formatMoney(m.expenses, currency)}
  </span>
  <span className="nums hidden flex-1 text-right text-muted-foreground sm:b
    +{formatMoney(m.income, currency)}
  </span>
  <span className={cn('nums w-24 shrink-0 text-right font-medium', m.net >=
    {formatMoney(m.net, currency)}
  </span>
  {showSpecial ? (
    <span className="nums hidden w-20 shrink-0 items-center justify-end gap
      {m.special > 0 ? (
        <>
          <Star className="h-3 w-3" aria-hidden /> {formatMoney(m.special,
        </>
      ) : null}
    </span>
  ) : null}
</Link>
  )}}
</div>
</CardContent>
</Card>
);
}

```

Mount at the bottom of pages/Analytics.jsx: `<MonthlyHistory series={data.series} currency={currency} showSpecial={!me?.preferences?.specialExpensesEnabled && data.series.some((s) => s.special > 0)} />` (a column-header row can be added above the list if it reads unclear in situ — implementer's call, keep it quiet).

- **Step 3: Deep-linkable month filter** — pages/Transactions.jsx: import `{ useSearchParams }` from 'react-router-dom'; then

```

const [searchParams] = useSearchParams();
const urlMonth = /^\\d{4}-\\d{2}$/.test(searchParams.get('month') ?? '') ? searchParams.g
const [monthFilter, setMonthFilter] = useState(urlMonth ?? 'all');

```

and make the list query month-aware so old months actually load (the page fetches max 200 recent rows today):

```
const { data, ... } = useQuery({
  queryKey: ['transactions', monthFilter],
  queryFn: () => api.get(monthFilter === 'all' ? '/api/transactions?limit=200' : `/api/
});
```

The month <Select> options (line ~269) must include urlMonth even when it's older than the derived list — add it to the months memo if missing.

- ☐ **Step 4: Verify** — dev server: Analytics shows the history table (current month marked “so far”, months before signup absent); tap an old month -> Transactions opens filtered to it and rows load (check a month older than the 200-row window on real data, or via mock seed); Special column appears only when the pref is on and a month has special spend. Client build passes.
- ☐ **Step 5: Docs + commit** — FEATURES.md Analytics section: history table; BUILD_PLAN tick 9.4. Commit 9.4: monthly spending history on Analytics + deep-linkable month filter.

Task 5: Encryption at rest (BUILD_PLAN 9.5) — LAST, biggest, riskiest

Read spec §3.5 first. Verified: all money math is JS-side; only categories.js:77 (ilike on description) and categories.js:100 (eq on category name) read sensitive content in SQL — both move to JS. **Do not start this task until 9.1–9.4 are merged and verified.**

Files: - Create: server/lib/crypto.js, server/test/crypto.test.js, server/migrations/012_encrypt, server/scripts/encrypt-backfill.mjs, server/migrations/013_encryption_drop_plaintext.sql
 - Modify: every route/lib that reads or writes the encrypted columns (routes/: transactions, budgets, dashboard, analytics, projections, affordability, wins, goals, categories, subscriptions, me, ask; lib/: askContext.js, subscriptions callers), server/package.json (test script), server/.env + Vercel env (DATA_ENCRYPTION_KEY), SECURITY.md, ARCHITECTURE.md schema notes, FEATURES.md, BUILD_PLAN.md

Interfaces: - Produces: encryptField(userId, plaintext) -> string (v1:<iv>:<tag>:<ct> base64 triple), decryptField(userId, stored) -> string (throws on tamper/wrong user), encryptAmount(userId, number) -> string, decryptAmount(userId, stored) -> number, from server/lib/crypto.js. - Encrypted columns (spec §3.5 table): transactions.amount+description; budgets.amount_limit; categories.name; savings_goals.name+target_amount+current_savings_contributions.amount+note; subscription_overrides.display_name; user_stats.monthly_limit; ask_messages.content.

- ☐ **Step 1: Key setup (Alex does this — exact instructions)** — openssl rand -base64 32, then add DATA_ENCRYPTION_KEY=<value> to server/.env, to Vercel (server project -> Settings -> Environment Variables), **and back it up in ~/Keys/trim-data-encryption-key.txt** — losing it destroys all user data irrecoverably.
- ☐ **Step 2: Failing tests first** — server/test/crypto.test.js:

```
import { test } from 'node:test';
import assert from 'node:assert/strict';
import { encryptField, decryptField, encryptAmount, decryptAmount } from '../lib/crypto
```

```

process.env.DATA_ENCRYPTION_KEY ??= Buffer.alloc(32, 7).toString('base64');
const USER_A = '00000000-0000-4000-8000-000000000001';
const USER_B = '00000000-0000-4000-8000-000000000002';

test('round-trips text', () => {
  const stored = encryptField(USER_A, 'Coffee with Em');
  assert.notEqual(stored, 'Coffee with Em');
  assert.ok(stored.startsWith('v1:'));
  assert.equal(decryptField(USER_A, stored), 'Coffee with Em');
});

test('round-trips amounts as numbers', () => {
  assert.equal(decryptAmount(USER_A, encryptAmount(USER_A, 123.45)), 123.45);
});

test('ciphertext is bound to the user', () => {
  const stored = encryptField(USER_A, 'secret');
  assert.throws(() => decryptField(USER_B, stored));
});

test('tampered ciphertext throws', () => {
  const stored = encryptField(USER_A, 'secret');
  const parts = stored.split(':');
  parts[3] = Buffer.from('tampered!').toString('base64');
  assert.throws(() => decryptField(USER_A, parts.join(':')));
});

test('unique IVs – same plaintext, different ciphertext', () => {
  assert.notEqual(encryptField(USER_A, 'same'), encryptField(USER_A, 'same'));
});

```

Add "test": "node --test test/" to server/package.json scripts. Run cd server && npm test — expect FAIL (module not found).

□ **Step 3: Implement** — server/lib/crypto.js:

```

/**
 * Phase 9.5 – at-rest encryption of users' financial data.
 * AES-256-GCM; per-user key derived from DATA_ENCRYPTION_KEY via HKDF so a
 * value copied into another user's row will not decrypt. Stored format:
 * v1:<iv b64>:<auth tag b64>:<ciphertext b64> (in text columns).
 * Losing DATA_ENCRYPTION_KEY = losing every user's data. See SECURITY.md.
 */
import { createCipheriv, createDecipheriv, hkdfSync, randomBytes } from 'node:crypto';

const VERSION = 'v1';
const HKDF_SALT = 'trim-data-v1';

```

```

function masterKey() {
  const raw = process.env.DATA_ENCRYPTION_KEY;
  if (!raw) throw new Error('DATA_ENCRYPTION_KEY is not set');
  const key = Buffer.from(raw, 'base64');
  if (key.length !== 32) throw new Error('DATA_ENCRYPTION_KEY must be 32 bytes base64!');
  return key;
}

function userKey(userId) {
  return Buffer.from(hkdfSync('sha256', masterKey(), HKDF_SALT, `user:${userId}`, 32));
}

export function encryptField(userId, plaintext) {
  if (plaintext === null || plaintext === undefined) return null;
  const iv = randomBytes(12);
  const cipher = createCipheriv('aes-256-gcm', userKey(userId), iv);
  const ct = Buffer.concat([cipher.update(String(plaintext), 'utf8'), cipher.final()]);
  return [VERSION, iv.toString('base64'), cipher.getAuthTag().toString('base64'), ct.toString('base64')];
}

export function decryptField(userId, stored) {
  if (stored === null || stored === undefined) return null;
  const [version, ivB64, tagB64, ctB64] = String(stored).split(':');
  if (version !== VERSION) throw new Error(`Unknown ciphertext version: ${version}`);
  const decipher = createDecipheriv('aes-256-gcm', userKey(userId), Buffer.from(ivB64, 'base64'));
  decipher.setAuthTag(Buffer.from(tagB64, 'base64'));
  return Buffer.concat([decipher.update(Buffer.from(ctB64, 'base64')), decipher.final()]);
}

export function encryptAmount(userId, amount) {
  return amount === null || amount === undefined ? null : encryptField(userId, String(amount));
}

export function decryptAmount(userId, stored) {
  const s = decryptField(userId, stored);
  return s === null ? null : Number(s);
}

```

Run `npm test` — all 5 pass.

□ Step 4: Migration 012 (additive only)

```

-- server/migrations/012_encryption_columns.sql
-- Phase 9.5 step 1 of 3: parallel encrypted columns. Plaintext dropped in 013
-- ONLY after encrypt-backfill.mjs verifies round-trips.
alter table public.transactions add column amount_enc text, add column description_enc text;
alter table public.budgets add column amount_limit_enc text;
alter table public.categories add column name_enc text;
alter table public.savings_goals add column name_enc text, add column target_amount text;

```

```

alter table public.savings_contributions add column amount_enc text, add column note_en
alter table public.subscription_overrides add column display_name_enc text;
alter table public.user_stats add column monthly_limit_enc text;
alter table public.ask_messages add column content_enc text;

```

Apply via MCP apply_migration (name encryption_columns).

- **Step 5: Backfill script** — server/scripts/encrypt-backfill.mjs: loads .env, iterates each table above in pages of 500 (select id/user_id + plaintext cols where <first_enc_col> is null), writes _enc values with encryptField/encryptAmount, then **re-reads and decrypt-verifies every row against the original, aborting loudly on any mismatch**; idempotent (the is null filter skips done rows); prints per-table counts. savings_contributions has no user_id? It does (schema: goal_id, user_id, amount...) — use it. ask_messages uses its user_id. Full script structure:

```

import 'dotenv/config';
import { createClient } from '@supabase/supabase-js';
import { encryptField, decryptField } from '../lib/crypto.js';

const supabase = createClient(process.env.SUPABASE_URL, process.env.SUPABASE_SERVICE_ROLE_KEY);

const JOBS = [
  { table: 'transactions', fields: [['amount', 'amount_enc'], ['description', 'description_enc']] },
  { table: 'budgets', fields: [['amount_limit', 'amount_limit_enc']] },
  { table: 'categories', fields: [['name', 'name_enc']] },
  { table: 'savings_goals', fields: [['name', 'name_enc'], ['target_amount', 'target_amount_enc']] },
  { table: 'savings_contributions', fields: [['amount', 'amount_enc'], ['note', 'note_enc']] },
  { table: 'subscription_overrides', fields: [['display_name', 'display_name_enc']], pk: ['user_id'] },
  { table: 'user_stats', fields: [['monthly_limit', 'monthly_limit_enc']], pk: ['user_id'] },
  { table: 'ask_messages', fields: [['content', 'content_enc']] },
];

for (const job of JOBS) {
  const pk = job.pk ?? ['id'];
  const plainCols = job.fields.map([p] => p);
  const firstEnc = job.fields[0][1];
  let done = 0;
  for (;;) {
    const { data: rows, error } = await supabase
      .from(job.table)
      .select([...pk, 'user_id', ...plainCols].filter((v, i, a) => a.indexOf(v) === i))
      .is(firstEnc, null)
      .limit(500);
    if (error) throw error;
    if (!rows.length) break;
    for (const row of rows) {
      const patch = {};
      for (const [plain, enc] of job.fields) {
        patch[enc] = row[plain] === null ? null : encryptField(row.user_id, String(row[plain]));
      }
      await supabase
        .from(job.table)
        .update(patch)
        .eq(pk, row[pk]);
    }
    done++;
  }
}

```

```

    }
    let q = supabase.from(job.table).update(patch);
    for (const k of pk) q = q.eq(k, row[k]);
    const { error: upErr } = await q;
    if (upErr) throw upErr;
    // verify round-trip immediately
    for (const [plain, enc] of job.fields) {
      if (row[plain] !== null && decryptField(row.user_id, patch[enc]) !== String(row[enc]))
        throw new Error(`VERIFY FAILED ${job.table} ${JSON.stringify(row)}`);
    }
  }
  done += 1;
}
}
console.log(`${job.table}: ${done} rows encrypted + verified`);
}
console.log('Backfill complete.');
```

Run: `cd server && node scripts/encrypt-backfill.mjs` -> per-table counts, no errors.
 Run again -> all zeros (idempotent).

- **Step 6: Route sweep** — switch reads/writes to the `_enc` columns via the crypto helpers. Pattern (POST `/api/transactions` insert): `amount_enc: encryptAmount(req.user.id, amount)`, `description_enc: description ? encryptField(req.user.id, description) : null` and after any select, map rows through a small per-route `decryptTx` helper before existing math. **Write both plaintext and `_enc` during this step** (dual-write) so the app works before AND after 013. Routes to touch and what they read: transactions (amount, description), dashboard (amounts + recent descriptions), budgets (amount_limit, amounts), analytics (amounts), projections (amounts), affordability (amounts, goal amounts, contribution amounts), wins (amounts, goal amounts/names, contribution amounts), goals (names, amounts; milestone math unchanged post-decrypt), categories (name + the suggest endpoint: replace `.ilike` with fetching the user's last 300 transactions' `description_enc`, decrypting, substring-matching term in JS; replace `.eq('name', keywordName)` with decrypt-and-compare over the user's category list), subscriptions (descriptions for detection, `display_name` override), me (monthly_limit), ask + askContext (content, amounts, descriptions, category names, goal fields). `lib/subscriptions.js` and `lib/gamification.js` stay pure — decrypt before calling them.

- **Step 7: Click-through on encrypted data** — dev server against real Supabase: log in as the test account, exercise Dashboard / Transactions (create, edit, star) / Budgets / Analytics / Goals (contribute) / Subscriptions / Ask Trim / Settings (change monthly limit) — everything renders decrypted values; Supabase Table Editor shows `v1:...` ciphertext in `_enc` columns. `npm run build` passes.

- **Step 8: Migration 013 (destructive — only after Step 7 passes and Alex confirms)**

```

-- server/migrations/013_encryption_drop_plaintext.sql
-- Phase 9.5 step 3 of 3. IRREVERSIBLE. Run only after backfill verification
-- AND a full click-through of the app reading _enc columns.
alter table public.transactions
```

```

drop column amount, drop column description;
alter table public.transactions rename column amount_enc to amount;
alter table public.transactions rename column description_enc to description;
alter table public.budgets drop column amount_limit;
alter table public.budgets rename column amount_limit_enc to amount_limit;
alter table public.categories drop column name;
alter table public.categories rename column name_enc to name;
alter table public.savings_goals drop column name, drop column target_amount, drop column current_amount;
alter table public.savings_goals rename column name_enc to name;
alter table public.savings_goals rename column target_amount_enc to target_amount;
alter table public.savings_goals rename column current_amount_enc to current_amount;
alter table public.savings_contributions drop column amount, drop column note;
alter table public.savings_contributions rename column amount_enc to amount;
alter table public.savings_contributions rename column note_enc to note;
alter table public.subscription_overrides drop column display_name;
alter table public.subscription_overrides rename column display_name_enc to display_name;
alter table public.user_stats drop column monthly_limit;
alter table public.user_stats rename column monthly_limit_enc to monthly_limit;
alter table public.ask_messages drop column content;
alter table public.ask_messages rename column content_enc to content;

```

Before applying: **categories** name had a uniqueness/seed dependency? Check migration 001 for constraints on dropped columns (e.g. name unique per user, amount CHECK constraints) and recreate none that reference plaintext values — ciphertext uniqueness is meaningless; drop such constraints in this migration. After applying: remove the dual-writes and the _enc suffixes from route code (columns now carry the original names with encrypted content), delete plaintext fallbacks. Re-run Step 7’s click-through.

Category-seeding trigger (spec §3.5 addendum): the handle_new_user SQL trigger seeds 12 default categories with plaintext name — it cannot encrypt because the key lives only in the server environment, never in Postgres. Resolution: move default-category seeding out of the trigger into the server — on GET /api/me, when the user has zero categories, seed the 12 defaults through the API path with encrypted names (mirrors the existing lazy user_stats insert at me.js:27–35). This migration drops the category-seeding portion of the trigger; user_stats seeding stays in SQL (nothing in that seed row is encrypted).

- **Step 9: SECURITY.md + docs + commit** — add the spec §3.5 “Honest limits” block verbatim to SECURITY.md; ARCHITECTURE.md schema section notes encrypted columns + DATA_ENCRYPTION_KEY; FEATURES.md gets a one-line “your data is encrypted at rest” note; BUILD_PLAN tick 9.5. Commit 9.5: encrypt financial data at rest (AES–256–GCM, per-user keys).

Self-review notes (done at plan time)

- Spec coverage: 3.1->Task 1, 3.2->Task 2, 3.3->Task 3, 3.4->Task 4 (+month param built in Task 2), 3.5->Task 5, §4 ordering preserved, §5 done-criteria in Global Constraints. Trigger/seeding conflict discovered during planning is resolved in Task 5 Step 8.

- Type consistency: `excludeSpecial(rows, enabled)` used identically in Tasks 2–3; `pace {target, spent, delta}` produced in Task 3 Step 1, consumed in Steps 3–4; `encryptField/decryptField/encryptAmount/decryptAmount` signatures match tests and backfill.
- Known judgement calls left to build sessions: exact placement/styling of the history column headers; whether `subscription_overrides.display_name` matching needs its `merchant_key` untouched (it does — keys are synthetic, not sensitive).